

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



DỮ LIỆU LỚN

Lab 4: Spark Streaming, Kafka và MongoDB

Nhóm thực hiện: CQ12

Phan Bá Đức	22120071
Đặng Duy Lâm	22120182
Nguyễn Quốc Thịnh	22120347
Trần Đức Thịnh	22120348

Giảng viên hướng dẫn:

TS. Nguyễn Ngọc Thảo

ThS. Huỳnh Lâm Hải Đăng

TP. Hồ Chí Minh, tháng 6 năm 2025.

Mục lục

Mục lục.....	1
1. Tổng quan.....	3
1.1. Phân công nhiệm vụ.....	3
1.2. Tóm tắt.....	3
2. Extract	3
2.1. Ý tưởng	3
2.2. Cơ chế hoạt động trong code	4
2.3. Logging.....	4
2.4. Cài đặt.....	4
2.5. Kết quả.....	6
3. Transform	7
3.1. transform_moving.py.....	7
3.1.1. Cấu trúc mã nguồn	7
3.1.2. Các thành phần chính	8
3.1.3. Phương pháp triển khai	9
3.2. transform_zscore.py.....	10
3.2.1. Cấu trúc mã nguồn	10
3.2.2. Các thành phần chính	10
3.2.3. Phương pháp triển khai	12
3.3. Cài đặt.....	13
3.4. Kết quả.....	14
4. Load.....	14
4.1. Cấu trúc chương trình	14
4.1.1. Hàm khởi tạo và cấu hình.....	14
4.1.2. Hàm đọc và xử lý dữ liệu	14
4.1.3. Hàm ghi dữ liệu.....	15
4.1.4. Hàm main	15
4.2. Cài đặt.....	15
4.3. Kết quả.....	17
5. Bonus.....	17
5.1. Giải thích mã nguồn.....	18

5.1.1.	Phương thức vận hành.....	18
5.2.	Thiết lập cá nhân ban đầu	19
5.3.	Cài đặt các thành phần cần thiết	20
5.3.1.	Cài đặt và cấu hình Java và Hadoop	20
5.3.2.	Cài đặt và cấu hình Kafka	20
5.3.3.	Cài đặt và cấu hình PySpark	21
5.4.	Khởi động hệ thống	22
5.4.1.	Khởi động HDFS và YARN trong Hadoop.....	22
5.4.2.	Chạy Kafka Zookeeper và Kafka Broker.....	23
5.4.3.	Tạo Kafka Topic cho high_price và low_price	24
5.4.4.	Chạy Pipeline dữ liệu	25
	Tài liệu tham khảo.....	28

1. Tổng quan

1.1. Phân công nhiệm vụ

MSSV	Họ và tên	Công việc	Hoàn thành
22120071	Phan Bá Đức	Extract	100%
22120182	Đặng Duy Lâm	Load	100%
22120347	Nguyễn Quốc Thịnh	Bonus	100%
22120348	Trần Đức Thịnh	Transform	100%

1.2. Tóm tắt

Báo cáo này trình bày quy trình xây dựng một pipeline ETL (Extract – Transform – Load) sử dụng streaming để xử lý dữ liệu giá Bitcoin (BTCUSDT) từ sàn Binance.

Bài tập này gồm ba giai đoạn chính:

- **Extract** để thu thập dữ liệu thông qua API Binance và đẩy vào Kafka với tần suất cao.
- **Transform** để tính toán trung bình trượt và điểm z-score theo nhiều sliding window bằng Spark Structured Streaming.
- **Load** để lưu dữ liệu đã xử lý vào MongoDB.
- Ngoài ra, có **bonus** cũng được thực hiện để xác định khoảng thời gian ngắn nhất trong 20 giây kế tiếp mà giá cao hơn hoặc thấp hơn giá hiện tại, rồi ghi vào hai topic Kafka mới (**btc-price-higher** và **btc-price-lower**).

Mỗi thành phần được triển khai dưới dạng module riêng biệt. Toàn bộ pipeline hoạt động liên tục, theo thời gian thực, cho phép khai thác nhanh chóng các tín hiệu biến động giá và phục vụ các ứng dụng phân tích tiếp theo.

2. Extract

Trong giai đoạn **Extract**, chương trình liên tục gọi API của Binance để lấy giá giao dịch của BTCUSDT, mỗi lần truy vấn nhận về cặp (*symbol*, *price*) sẽ được bổ sung trường thời gian chuẩn ISO 8601 (UTC) để theo dõi thời gian nhận được thông tin đó.

Từ ba trường thông tin vừa nhận được, tạo một record JSON và đẩy vào Kafka với topic **btc-price** với tần suất ~10 record/s, bảo đảm luồng dữ liệu luôn cập nhật thời gian thực. Đồng thời, toàn bộ quá trình, từ việc gọi API, cho đến kết quả gửi vào Kafka, đều được ghi chi tiết vào file `extract.log`, để dễ dàng theo dõi và debug.

2.1. Ý tưởng

Để cải thiện hiệu suất và giảm độ trễ trong quá trình truy vấn giá BTC từ Binance, nhóm sẽ triển khai hai luồng xử lý song song, gọi là **fetch threads**, nhằm đồng thời thực hiện các yêu cầu API. Mỗi luồng sau khi nhận dữ liệu sẽ đẩy bản ghi vào một hàng đợi, trong khi luồng chính (**main thread**) chịu trách nhiệm lấy các bản ghi từ hàng đợi này và gửi vào Kafka.

Thiết kế phân chia nhiệm vụ như vậy giúp tận dụng tốt tài nguyên hệ thống và duy trì tốc độ ghi dữ liệu ổn định. Ngoài ra, hệ thống cũng tái sử dụng kết nối HTTP thông qua đối tượng requests.Session, góp phần giảm chi phí thiết lập kết nối mới, từ đó tối ưu hóa hiệu năng và độ tin cậy của quá trình thu thập dữ liệu.

2.2. Cơ chế hoạt động trong code

Nhóm em tiến hành fetch data trên 2 luồng song song, với tần suất mỗi luồng là 1 request/100ms. Khi đó code sẽ chạy, tiến hành lấy dữ liệu từ API với tần suất 20 requests/giây (tức 2 requests mỗi 100ms). Tuy nhiên vì cách thao tác xử lý dữ liệu và độ trễ của mạng. Số request thu được không đảm bảo lúc nào cũng là 20 requests/giây mà sẽ giao động từ 10 - 20 requests (trong 1 số trường hợp đặc biệt, số request có thể dưới 10, nhưng điều này rất hiếm khi xảy ra, nếu xảy ra lý do lớn nhất là vì mạng người dùng lúc đó đang gặp sự cố).

Vì thế nhóm em quyết định làm tròn timestamp theo tần suất 1request/100ms, tức là làm tròn xuống theo s.s00

Quá trình này sẽ được ghi log vào file extract.log, chứa thông tin về mỗi record (thông tin lấy về bằng API và timestamp tương ứng) được tạo, và record đó do luồng nào lấy về (có 2 luồng), record nào đã được gửi đến kafka và số lượng record đã gửi đến kafka trong 1 giây vừa rồi.

Khi dừng chương trình (Ctrl + C), dùng `stop_event` và xử lý `KeyboardInterrupt` để dừng threads, đóng tài nguyên (producer, session, logging).

```
# Cấu hình API và Kafka
API_ENDPOINT = "https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT"
FREQUENCY_PER_100MS = 1 # 1 lần/100ms mỗi luồng
SLEEP_TIME = 0.1 / FREQUENCY_PER_100MS # 0.1 giây
NUM_THREADS = 2 # Số luồng fetch data
```

Một vài setting mặc định trong code.

2.3. Logging

Nhóm sẽ sử dụng file log thay vì console để dễ kiểm tra sau, đặc biệt khi chạy lâu. Vì khi xuất ra nhiều dữ liệu như vậy, console sẽ cắt bớt 1 phần dữ liệu gây khó theo dõi. Trong khi Logging ghi lại mọi thứ vào file, giúp debug và theo dõi hiệu suất (như tần suất record/giây, lỗi API).

2.4. Cài đặt

Đầu tiên, nhóm em sẽ tiến hành cài đặt kafka và cấu hình một số file cần thiết để có thể chạy được. Trước hết là tải file cài đặt kafka về:

```

base 11:28:57
wget https://downloads.apache.org/kafka/3.9.1/kafka_2.13-3.9.1.tgz
--2025-06-06 11:29:05-- https://downloads.apache.org/kafka/3.9.1/kafka_2.13-3.9.1.tgz
Resolving downloads.apache.org (downloads.apache.org)... 2a01:4f9:3a:2c57::2, 2a01:4f8:10a:39da::2, 88.99.208.237, ...
Connecting to downloads.apache.org (downloads.apache.org)|2a01:4f9:3a:2c57::2|:443... failed: Network is unreachable.
Connecting to downloads.apache.org (downloads.apache.org)|2a01:4f8:10a:39da::2|:443... failed: Network is unreachable.
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 122110298 (116M) [application/x-gzip]
Saving to: 'kafka_2.13-3.9.1.tgz'

kafka_2.13-3.9.1.tg 100%[=====] 116,45M 7,45MB/s in 22s S
2025-06-06 11:29:30 (5,32 MB/s) - 'kafka_2.13-3.9.1.tgz' saved [122110298/122110298]

```

Tải kafka

Sau đó, tiến hành giải nén và thực hiện cấp quyền cho các folder:

```

base 11:29:30
sudo tar -xzf kafka_2.13-3.9.1.tgz -C /opt
kafka_2.13-3.9.1/
kafka_2.13-3.9.1/LICENSE
kafka_2.13-3.9.1/NOTICE
kafka_2.13-3.9.1/bin/
kafka_2.13-3.9.1/bin/kafka-delete-records.sh
kafka_2.13-3.9.1/bin/trogdor.sh
kafka_2.13-3.9.1/bin/kafka-jmx.sh
kafka_2.13-3.9.1/bin/connect-mirror-maker.sh
kafka_2.13-3.9.1/bin/kafka-console-consumer.sh
kafka_2.13-3.9.1/bin/kafka-consumer-perf-test.sh
kafka_2.13-3.9.1/bin/kafka-log-dirs.sh
kafka_2.13-3.9.1/bin/kafka-metadata-quorum.sh
kafka_2.13-3.9.1/bin/zookeeper-server-stop.sh
kafka_2.13-3.9.1/bin/kafka-verifiable-consumer.sh
kafka_2.13-3.9.1/bin/kafka-features.sh

```

Giải nén file vừa tải

```

base 11:31:01
sudo mv /opt/kafka_2.13-3.9.1 /opt/kafka

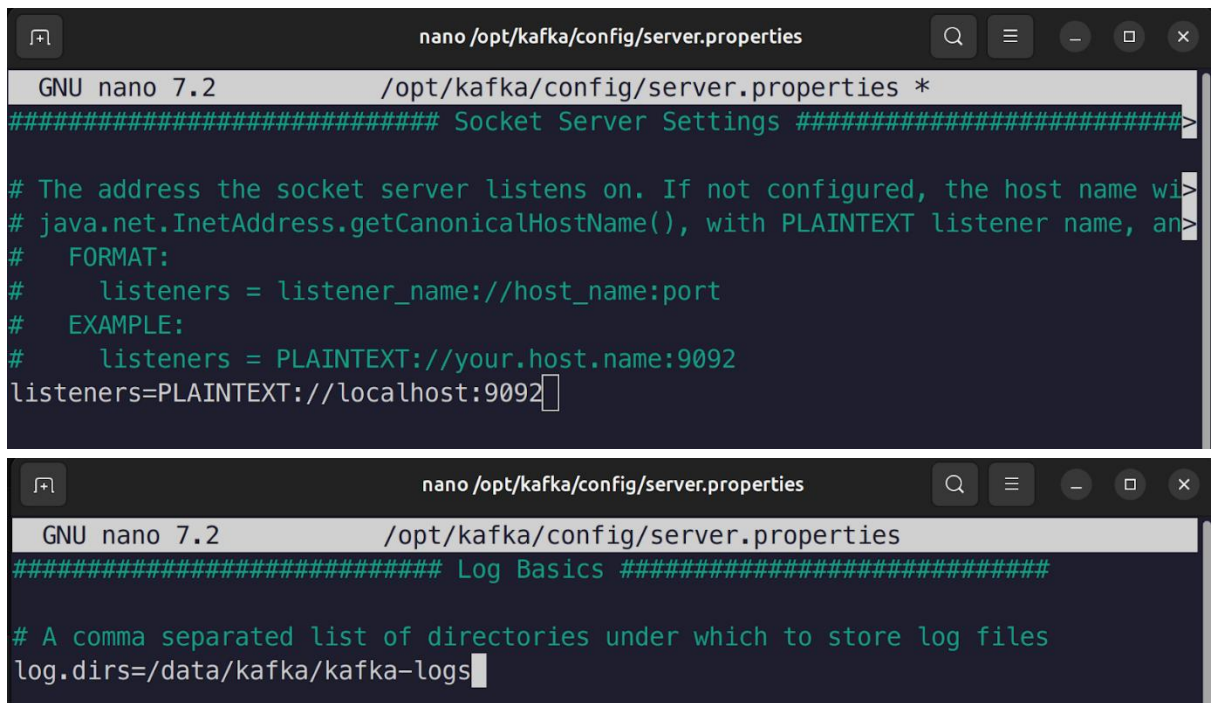
base 11:39:11
sudo mkdir -p /data/kafka

base 11:40:33
sudo chown $USER -R /opt/kafka /data/kafka

```

Đổi tên và cấp quyền cho lại cho các folder

Và cuối cùng là cập nhật lại các cài đặt cho server của kafka trong *server.properties*:



The first screenshot shows the 'Socket Server Settings' section of the `/opt/kafka/config/server.properties` file. The `listeners` property is being set to `PLAINTEXT://localhost:9092`. The second screenshot shows the 'Log Basics' section where the `log.dirs` property is being set to `/data/kafka/kafka-logs`.

```

nano /opt/kafka/config/server.properties
GNU nano 7.2 /opt/kafka/config/server.properties *
##### Socket Server Settings #####
# The address the socket server listens on. If not configured, the host name will be
# java.net.InetAddress.getCanonicalHostName(), with PLAINTEXT listener name, and
# FORMAT:
#   listeners = listener_name://host_name:port
# EXAMPLE:
#   listeners = PLAINTEXT://your.host.name:9092
listeners=PLAINTEXT://localhost:9092

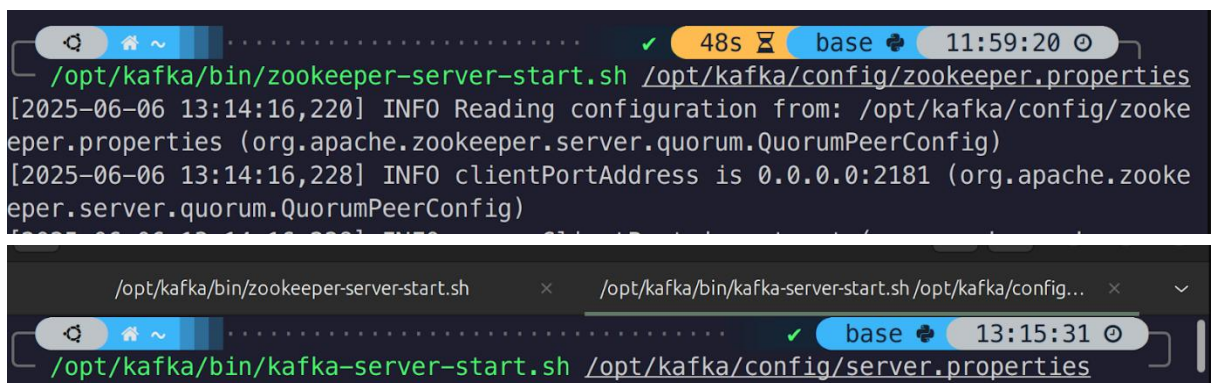
nano /opt/kafka/config/server.properties
GNU nano 7.2 /opt/kafka/config/server.properties
##### Log Basics #####
# A comma separated list of directories under which to store log files
log.dirs=/data/kafka/kafka-logs
  
```

Cập nhật một vài properties cho kafka server

Vậy là bước chuẩn bị đã xong.

2.5. Kết quả

Đầu tiên, chạy zookeeper và kafka:



The first screenshot shows the execution of `/opt/kafka/bin/zookeeper-server-start.sh` with output logs indicating successful configuration reading. The second screenshot shows the execution of `/opt/kafka/bin/kafka-server-start.sh`.

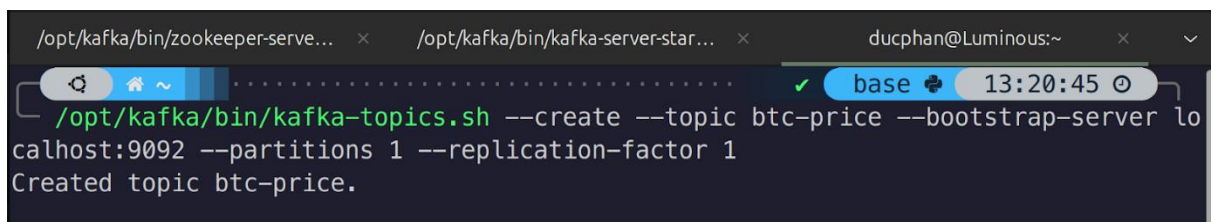
```

/opt/kafka/bin/zookeeper-server-start.sh /opt/kafka/config/zookeeper.properties
[2025-06-06 13:14:16,220] INFO Reading configuration from: /opt/kafka/config/zookeeper.properties (org.apache.zookeeper.server.quorum.QuorumPeerConfig)
[2025-06-06 13:14:16,228] INFO clientPortAddress is 0.0.0.0:2181 (org.apache.zookeeper.server.quorum.QuorumPeerConfig)

/opt/kafka/bin/zookeeper-server-start.sh
/opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/server.properties
  
```

Chạy zookeeper và kafka

Tiếp theo, mở 1 terminal khác và tạo 1 topic tên **btc-price**:



The screenshot shows the execution of `/opt/kafka/bin/kafka-topics.sh` with flags to create a new topic named `btc-price` with 1 partition and a replication factor of 1.

```

/opt/kafka/bin/kafka-topics.sh --create --topic btc-price --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
Created topic btc-price.
  
```

Tạo topic btc-price

Cuối cùng, chạy chương trình `extract.py` bằng lệnh: `python extract.py` và mở file log để xem những record được gửi đến kafka:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.100000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.100000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.200000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.200000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.300000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.300000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.400000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.400000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.500000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.500000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.600000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.600000Z"}
{"symbol": "BTCUSD", "price": 105399.44, "timestamp": "2025-06-09T06:05:11.700000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.700000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.800000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.900000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:11.900000Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:12Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:12Z"}
{"symbol": "BTCUSD", "price": 105399.45, "timestamp": "2025-06-09T06:05:12.100000Z"}

```

Log file

3. Transform

Ở giai đoạn **transformation**, yêu cầu khởi tạo 2 chương trình `transform_moving` và `transform_zscore` thực hiện 2 tác vụ chính, được nhóm thực hiện trong 2 file sau:

- **transform_moving.py**: Là chương trình xử lý luồng dữ liệu sử dụng Apache Spark Structured Streaming để đọc dữ liệu giá Bitcoin từ Topic `btc-price`, tính toán giá trung bình và độ lệch chuẩn theo các cửa sổ thời gian trượt (sliding windows), và xuất kết quả ra một topic `btc-price-moving`.

- **transform_zscore.py**: sử dụng PySpark để xử lý dữ liệu thời gian thực từ Kafka. Mục tiêu chính của mã là đọc dữ liệu giá Bitcoin từ hai topic Kafka (**btc-price** và **btc-price-moving**), thực hiện phép tính Z-score dựa trên giá và các giá trị trung bình, độ lệch chuẩn, sau đó ghi kết quả vào một topic `btc-price-zscore`.

3.1. transform_moving.py

3.1.1. Cấu trúc mã nguồn

Cấu trúc tổng thể bao gồm:

- Hàm khởi tạo và cấu hình:
 - `create_schema_window()`: Định nghĩa schema cho dữ liệu và các window.
 - `create_SparkSession()`: Khởi tạo SparkSession với các cấu hình cần thiết.
- Hàm đọc và xử lý dữ liệu:
 - `read_from_kafka()`: Đọc luồng dữ liệu từ topic Kafka (`btc-price`).
 - `create_parsed_df()`: Phân tích dữ liệu Kafka theo schema định nghĩa.
- Hàm xử lý logic chính:
 - `process_window_df()`: Tính toán giá trung bình và độ lệch chuẩn cho một window cụ thể.
 - `create_format_output()`: Tạo định dạng đầu ra cuối cùng bằng cách kết hợp các kết quả từ nhiều window.
- Hàm xuất dữ liệu:
 - `write_to_kafka()`: Ghi dữ liệu đã xử lý vào topic Kafka (`btc-price-moving`).

3.1.2. Các thành phần chính

Schema và Windows:

- Schema (**create_schema_window**):
 - Dữ liệu đầu vào từ Kafka có cấu trúc: timestamp (thời gian), symbol (mã tài sản, ví dụ: BTC), và price (giá).
 - Sử dụng StructType để định nghĩa schema với các kiểu dữ liệu cụ thể: TimestampType, StringType, DoubleType.
- Windows:
 - Định nghĩa các window: 30 giây, 1 phút, 5 phút, 15 phút, 30 phút, 1 giờ.
 - Mỗi window có thời gian trượt (sliding interval) tương ứng, ví dụ: window 30 giây trượt mỗi 6 giây.

SparkSession:

- Được khởi tạo với các cấu hình:
 - Tên ứng dụng: KafkaSparkStreaming.
 - Đường dẫn checkpoint để lưu trạng thái xử lý luồng.
 - Gói phụ thuộc Kafka (spark-sql-kafka-0-10_2.12:3.5.0).
 - Tắt kiểm tra tính đúng đắn của toán tử trạng thái để tối ưu hiệu suất.
- Mức log được đặt ở ERROR để giảm thiểu đầu ra console.

Đọc dữ liệu từ Kafka:

- Sử dụng readStream với định dạng Kafka.
- Cấu hình:
 - Server Kafka: localhost:9092.
 - Topic: *btc-price*.
 - Đọc từ offset sớm nhất (startingOffsets: earliest).
 - Giới hạn 100 bản ghi mỗi trigger (maxOffsetsPerTrigger).

Phân tích dữ liệu:

- Hàm **create_parsed_df** chuyển đổi dữ liệu Kafka (ở định dạng JSON) thành DataFrame với các cột: timestamp, symbol, price.
- Sử dụng from_json để parse chuỗi JSON theo schema đã định nghĩa.

Xử lý sliding window:

- Hàm **process_window_df**:
 - Áp dụng watermark (10 giây) để xử lý dữ liệu trễ.
 - Nhóm dữ liệu theo window và symbol.
 - Tính toán:
 - Giá trung bình (avg_price) bằng avg.
 - Độ lệch chuẩn (std_price) bằng stddev, thay thế giá trị null bằng 0.
 - Đầu ra có các cột: timestamp (thời điểm kết thúc window), symbol, window (tên window), avg_price, std_price.

Định dạng đầu ra:

- Hàm **create_format_output**:

- Kết hợp các DataFrame từ tất cả các window bằng union.
- Nhóm theo timestamp và symbol, thu thập danh sách các window (windows) dưới dạng cấu trúc JSON với các trường: window, avg_price, std_price.

Ghi dữ liệu vào kafka:

- Hàm **write_to_kafka**:
 - Chuyển DataFrame thành chuỗi JSON với cột value (yêu cầu của Kafka).
 - Ghi luồng vào topic **btc-price-moving** với chế độ **update** (dữ liệu giá Bitcoin được tính toán qua các cửa sổ trượt có thể thay đổi do thêm dữ liệu mới vào cùng cửa sổ nên sử dụng mode **update** để cập nhật khi xảy ra thay đổi)..
 - Sử dụng `awaitTermination` để giữ luồng chạy liên tục.

3.1.3. Phương pháp triển khai

Công nghệ sử dụng:

- Apache Spark Structured Streaming: Xử lý dữ liệu luồng theo thời gian thực.
- Kafka: Hệ thống hàng đợi phân tán để truyền dữ liệu đầu vào và đầu ra.
- PySpark: API Python của Spark để lập trình.

Quy trình xử lý:

1. Khởi tạo: Tạo `SparkSession` và định nghĩa schema, window.
2. Đọc dữ liệu: Đọc luồng từ topic Kafka **btc-price**.
3. Phân tích: Chuyển dữ liệu thô thành DataFrame có cấu trúc.
4. Xử lý:
 - Tính toán giá trung bình và độ lệch chuẩn cho từng window.
 - Kết hợp kết quả từ các window vào một DataFrame duy nhất.
5. Xuất dữ liệu: Ghi kết quả vào topic Kafka **btc-price-moving**.

Tối ưu hóa:

- Watermark: Xử lý dữ liệu trễ với ngưỡng 10 giây.
- Checkpointing: Lưu trạng thái để đảm bảo tính nhất quán khi gặp lỗi.
- Giới hạn offset: Giới hạn số bản ghi mỗi trigger để kiểm soát tải.
- Tắt kiểm tra toán tử trạng thái: Tăng hiệu suất nhưng cần đảm bảo logic đúng.

Định dạng đầu ra:

- Dữ liệu đầu ra có cấu trúc json:

```
{
  "timestamp": "2025-06-13T13:18:00.000+70:00",
  "symbol": "BTC",
  "windows": [
    {"window": "30s", "avg_price": 50000.0, "std_price": 100.0},
    {"window": "1m", "avg_price": 50010.0, "std_price": 120.0},
    ...
  ]
}
```

3.2. transform_zscore.py

3.2.1. Cấu trúc mã nguồn

Cấu trúc tổng thể bao gồm:

- Hàm khởi tạo và cấu hình:
 - `create_schema()`: Định nghĩa schema cho dữ liệu từ hai topic Kafka (btc-price và btc-price-moving).
 - `create_SparkSession()`: Khởi tạo SparkSession với các cấu hình cần thiết cho Spark Streaming và Kafka.
- Hàm đọc và xử lý dữ liệu:
 - `read_from_kafka()`: Đọc luồng dữ liệu từ các topic Kafka (btc-price và btc-price-moving).
 - `create_extract_parsed_df()`: Phân tích dữ liệu từ topic btc-price theo schema định nghĩa.
 - `create_moving_parsed_df()`: Phân tích và explode dữ liệu từ topic btc-price-moving.
- Hàm xử lý logic chính:
 - `create_join_df()`: Thực hiện inner join giữa hai DataFrame để kết hợp giá thực tế và thông kê giá.
 - `create_zscore_df()`: Tính toán Z-score dựa trên giá, giá trung bình, và độ lệch chuẩn.
 - `create_format_output()`: Định dạng đầu ra cuối cùng bằng cách nhóm dữ liệu và tạo danh sách Z-score cho các window.
- Hàm xuất dữ liệu:
 - `write_to_kafka()`: Ghi dữ liệu đã xử lý vào topic Kafka (btc-price-zscore).

3.2.2. Các thành phần chính

Định nghĩa Schema:

- Schema (create_schema):
 - SCHEMA_EXTRACT (cho topic **btc-price**):
 - Cấu trúc: timestamp (TimestampType), symbol (StringType), price (DoubleType).
 - Đại diện cho dữ liệu giá Bitcoin thời gian thực.
 - SCHEMA_MOVING (cho topic **btc-price-moving**):
 - Cấu trúc: timestamp (TimestampType), symbol (StringType), windows (ArrayType chứa các struct với window (StringType), avg_price (DoubleType), std_price (DoubleType)).
 - Đại diện cho thống kê giá (trung bình và độ lệch chuẩn) trên các cửa sổ thời gian.
 - Sử dụng StructType và ArrayType để hỗ trợ dữ liệu JSON phức tạp từ Kafka.

SparkSession:

- Được khởi tạo trong `create_SparkSession` với các cấu hình:

- Tên ứng dụng: KafkaSparkStreaming.
- Đường dẫn checkpoint: ./checkpoint để lưu trạng thái xử lý luồng.
- Gói phụ thuộc: spark-sql-kafka-0-10_2.12:3.5.0 để tích hợp Kafka.
- Tắt kiểm tra tính đúng đắn của toán tử trạng thái (statefulOperator.checkCorrectness.enabled = false) để tối ưu hiệu suất.
- Mức log: ERROR để giảm thiểu đầu ra console.

Đọc dữ liệu từ Kafka:

- Hàm **read_from_kafka**:
 - Sử dụng readStream với định dạng Kafka.
 - Cấu hình:
 - Server Kafka: localhost:9092.
 - Topic: **btc-price** và **btc-price-moving**.
 - Đọc từ offset sớm nhất (startingOffsets: earliest).
 - Giới hạn 200 bản ghi mỗi trigger (maxOffsetsPerTrigger).
 - Đầu ra là DataFrame chứa dữ liệu thô từ Kafka.

Phân tích dữ liệu:

- Hàm **create_extract_parsed_df**:
 - Parse dữ liệu từ topic btc-price thành DataFrame với các cột: timestamp, symbol, price.
 - Sử dụng from_json để chuyển đổi chuỗi JSON theo SCHEMA_EXTRACT.
 - Thêm watermark (10 giây) để xử lý dữ liệu trễ.
- Hàm **create_moving_parsed_df**:
 - Parse dữ liệu từ topic **btc-price-moving** thành DataFrame.
 - Thực hiện explode trên cột windows để tạo các hàng riêng biệt với các cột: timestamp, symbol, window, avg_price, std_price.
 - Thêm watermark (10 giây).

Xử lý logic chính:

- Hàm **create_join_df**:
 - Thực hiện inner join giữa hai DataFrame (extract_parsed_df và moving_parsed_df) dựa trên timestamp và symbol.
 - Đầu ra chứa các cột: timestamp, symbol, window, price, avg_price, std_price.
- Hàm **create_zscore_df**:
 - Tính Z-score của dữ liệu price.
 - Xử lý trường hợp std_price nhỏ hơn 0.00001 bằng cách gán Z-score là 0.
 - Đầu ra chứa các cột: timestamp, symbol, window, zscore_price.
- Hàm **create_format_output**:
 - Nhóm dữ liệu theo timestamp, symbol, window và tính trung bình zscore_price để xử lý bản ghi trùng lặp.
 - Nhóm lại theo timestamp, symbol và thu thập danh sách zscores dưới dạng mảng các struct (window, zscore_price).

Ghi dữ liệu vào Kafka:

- Hàm `write_to_kafka`:
 - Chuyển DataFrame thành chuỗi JSON với cột value (yêu cầu của Kafka).
 - Ghi luồng vào topic **btc-price-zscore** với chế độ **append** (đảm bảo rằng mỗi danh sách Z-score của cặp timestamp-symbol chỉ được ghi một lần vào topic **btc-price-zscore**, sau khi tất cả dữ liệu liên quan trong phạm vi watermark đã được xử lý).
 - Sử dụng `awaitTermination` để giữ luồng chạy liên tục.

3.2.3. Phương pháp triển khai

Công nghệ sử dụng:

- Apache Spark Structured Streaming: Xử lý dữ liệu luồng thời gian thực với hiệu suất cao.
- Kafka: Hệ thống hàng đợi phân tán để truyền dữ liệu đầu vào (**btc-price**, **btc-price-moving**) và đầu ra (**btc-price-zscore**).
- PySpark: API Python của Spark để lập trình và xử lý dữ liệu.

Quy trình xử lý:

1. Khởi tạo: Tạo SparkSession và định nghĩa schema cho dữ liệu Kafka.
2. Đọc dữ liệu: Đọc luồng từ hai topic Kafka (**btc-price** và **btc-price-moving**).
3. Phân tích: Chuyển dữ liệu thô thành DataFrame có cấu trúc, explode dữ liệu windows từ topic **btc-price-moving**.
4. Xử lý:
 - Thực hiện inner join để kết hợp giá thực tế với thống kê giá.
 - Tính Z-score để chuẩn hóa giá.
 - Định dạng đầu ra thành mảng zscores chứa thông tin Z-score cho từng window.
5. Xuất dữ liệu: Ghi kết quả vào topic Kafka **btc-price-zscore**.

Tối ưu hóa:

- Watermark: Sử dụng ngưỡng 10 giây để xử lý dữ liệu trễ.
- Checkpointing: Lưu trạng thái xử lý tại `./checkpoint` để đảm bảo tính nhất quán khi gặp lỗi.
- Giới hạn offset: Giới hạn 200 bản ghi mỗi trigger để kiểm soát tải xử lý.
- Tắt kiểm tra toàn tử trạng thái: Tăng hiệu suất, nhưng cần đảm bảo logic xử lý đúng.

Định dạng đầu ra:

- Dữ liệu đầu ra có cấu trúc json:

```
{
  "timestamp": "2025-06-15T19:21:00.000+07:00",
  "symbol": "BTC",
  "zscores": [
    {"window": "30s", "zscore_price": 1.5},
    {"window": "1m", "zscore_price": 1.2},
    ...
  ]
}
```

$$\left. \begin{array}{l} \text{ } \\ \text{ } \end{array} \right\} \left. \begin{array}{l} \text{ } \\ \text{ } \end{array} \right\}$$

3.3. Cài đặt

Đầu tiên, nhóm em sẽ thực hiện cài đặt công cụ Spark:

```
adrian@hadoop:~$ wget https://downloads.apache.org/spark/spark-3.5.6/spark-3.5.6-bin-hadoop3.tgz
--2025-06-11 22:43:58-- https://downloads.apache.org/spark/spark-3.5.6/spark-3.5.6-bin-hadoop3.tgz
Resolving downloads.apache.org (downloads.apache.org)... 2a01:4f8:10a:39da::2, 2a01:4f9:3a:2c57::2, 135.181.214.104, ...
Connecting to downloads.apache.org (downloads.apache.org)|2a01:4f8:10a:39da::2|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 400923510 (382M) [application/x-gzip]
Saving to: 'spark-3.5.6-bin-hadoop3.tgz'

spark-3.5.6-bin-hadoop3.tgz      100%[=====] 382.35M  7.42MB/s

2025-06-11 22:44:56 (6.73 MB/s) - 'spark-3.5.6-bin-hadoop3.tgz' saved [400923510/400923510]
```

```
tdthinh@tdthinh:~$ tar -xvzf spark-3.5.6-bin-hadoop3.tgz
spark-3.5.6-bin-hadoop3/
spark-3.5.6-bin-hadoop3/examples/
spark-3.5.6-bin-hadoop3/examples/jars/
spark-3.5.6-bin-hadoop3/examples/jars/spark-examples_2.12-3.5.6.jar
spark-3.5.6-bin-hadoop3/examples/jars/scopt_2.12-3.7.1.jar
spark-3.5.6-bin-hadoop3/examples/src/
```

Tải và giải nén Spark

Sau đó, thực hiện config cho **SPARK HOME** bằng cách thêm path vào `~/.bashrc`:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

Config cho SPARK HOME

Cuối cùng, kiểm tra cài đặt bằng cách chạy thử spark-shell:

```
tdthinh@tdthinh:~$ spark-shell
25/06/11 22:55:35 WARN Utils: Your hostname, tdthinh resolves to a local IP address; your Spark configuration should explicitly define a hostnames to avoid confusion.
25/06/11 22:55:35 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another IP.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel().
25/06/11 22:55:38 WARN NativeCodeLoader: Unable to load native-hadoop library for optimization
Spark context Web UI available at http://192.168.1.99:4040
Spark context available as 'sc' (master = local[*], app id = local-174)
Spark session available as 'spark'.
Welcome to

      /__\
     /\  \/ \_\/\_\/\_\/\_\/\_\/\_\/
    /___/ .__/\_\_,/_/_/_/_/_/_/_
   /__\

version 3.5.6

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.27)
```

Chay thử spark-shell

4.1.3. Hàm ghi dữ liệu

write_to_mongo(): Nhận một DataFrame và khởi tạo writeStream với format "mongodb". Hàm đặt URI kết nối mongodb://mongodb:27017, chọn database **btc**, collection **btc-price-zscore** và checkpoint: **“./chk/load/”** để bảo đảm việc ghi append liên tục, có khả năng khôi phục nếu job gián đoạn.

4.1.4. Hàm main

main(): hàm này sẽ thực hiện các công việc sau:

1. Tạo **SparkSession** và đặt log ở mức ERROR để tối thiểu thông tin được xuất ra terminal.
2. Đọc topic **btc-price-zscore** bằng **read_kafka_stream**.
3. Gọi **parse_and_explode** để phân phối các zscore vào các trường khác (timestamp, symbol, window).
4. Ghi dữ liệu vào database với hàm **write_to_mongo**.

4.2. Cài đặt

Do ở lab 1 em cài đặt môi trường hadoop ở trong docker, cho nên để cho dễ, ở phần này em cũng sẽ setup mongodb trong docker và cho chúng giao tiếp với nhau bằng cách thêm vào chung 1 network.

Đầu tiên, pull image của mongodb từ docker hub:

```
> docker pull mongodb/mongodb-community-server:latest
latest: Pulling from mongodb/mongodb-community-server
215ed5a63843: Pull complete
8ff8a41bb225: Pull complete
25ad53811bb7: Pull complete
1923b42181b4: Pull complete
df9fcfa822cd: Pull complete
690ac532328d: Pull complete
4ddba10d83da: Pull complete
58c6070c6753: Pull complete
04b60025e8d7: Pull complete
4f4fb700ef54: Pull complete
c0054b9f8e32: Pull complete
Digest: sha256:e6bdd5fd5ddceca75ea1b95df8c8f2c12e8cdd91f207b76764732111c7b5bab
Status: Downloaded newer image for mongodb/mongodb-community-server:latest
docker.io/mongodb/mongodb-community-server:latest
```

Pull image của mongodb

Tiếp theo, tạo 1 docker network để cho 2 container (1 từ môi trường em cài ở lab 1, 1 host mongodb) có thể giao tiếp với nhau:

```
> docker network create big_data_labs
38c575e12c464e26e7ccadd168c30b7b7e2956a128e8fc44d5196824747f4641
```

Tạo network big_data_labs

Sau đó, tạo container từ image của mongodb vừa pull về, đồng thời thêm nó vào network vừa tạo:

```
(miniconda3) ~
> docker run --name mongodb --network big_data_labs -p 27017:27017 -d mongodb/mongodb-community-server:latest
dcd45b0d493cd0e24d4343380cfdce95506e58c28cd61f385f00778f0e3b1c5c

(miniconda3) ~
> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
dcd45b0d493c   mongodb/mongodb-community-server:latest  "python3 /usr/local/..."  3 seconds ago  Up 3 seconds  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
1239540e0f0b   silicoflare/hadoop:amd               "/bin/bash -c bash"       2 months ago  Up 2 hours    0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp, 0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp, 9000/tcp
```

Chạy container host mongodb

Ngoài ra, do ban đầu container em tạo ra ở lab 1 chưa được thêm vào network mới tạo cho nên bây giờ em phải thêm vào bằng lệnh:

`docker network connect big_data_labs SRN`

Sau đó, kiểm tra network của 2 container:

```
(miniconda3) ~
> docker network inspect big_data_labs
[
  {
    "Name": "big_data_labs",
    "Id": "38c575e12c464e26e7ccadd168c30b7b7e2956a128e8fc44d5196824747f4641",
    "Created": "2025-06-15T13:18:21.157793151+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "1239540e0f0b1ad25b665ea5cc84b31020e9b388c16e9115b0c4d9af04696e35": {
        "Name": "SRN",
        "EndpointID": "de81e7f75db2475bb75fcf81329226d5b12f5783bfe1c47a83cc7b4732e6f600",
        "MacAddress": "0a:0c:03:b7:27:33",
        "IPv4Address": "172.20.0.3/16",
        "IPv6Address": ""
      },
      "dcd45b0d493cd0e24d4343380cfdce95506e58c28cd61f385f00778f0e3b1c5c": {
        "Name": "mongodb",
        "EndpointID": "0a7ba28021667866776bd2d72f94ead87a38239b9d92c3f12dcf67c0a79bd5b5",
        "MacAddress": "72:0f:16:7e:20:de",
        "IPv4Address": "172.20.0.2/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {}
  }
]
```

Kiểm tra network của 2 container

Vậy là đã xong bước setup cho mongodb.

4.3. Kết quả

Sau khi chạy các bước extract và load như ở trên, thực hiện chạy file load.py, kết quả nhận được như sau:

```
> docker exec -it mongodb mongosh
Current Mongosh Log ID: 68514db13c2fbc5cb9c59f34
Connecting to: mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.1
Using MongoDB: 8.0.9
Using Mongosh: 2.5.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

```
test> use btc
switched to db btc
btc> show collections
btc-price-zscore
btc> db['btc-price-zscore'].find({}, { _id: 0 }).sort({ timestamp: -1 }).limit(5).pretty()
[
  {
    timestamp: ISODate('2025-06-15T05:17:18.000Z'),
    symbol: 'BTCUSDT',
    window: '30s',
    zscore_price: 1.4780818838492826
  },
  {
    timestamp: ISODate('2025-06-15T05:17:12.000Z'),
    symbol: 'BTCUSDT',
    window: '30s',
    zscore_price: -0.33416791109699245
  },
  {
    timestamp: ISODate('2025-06-15T05:17:12.000Z'),
    symbol: 'BTCUSDT',
    window: '1m',
    zscore_price: -1.527286929276806
  },
  {
    timestamp: ISODate('2025-06-15T05:17:06.000Z'),
    symbol: 'BTCUSDT',
    window: '30s',
    zscore_price: -1.9296720573785942
  },
  {
    timestamp: ISODate('2025-06-15T05:17:00.000Z'),
    symbol: 'BTCUSDT',
    window: '30s',
    zscore_price: -1.2213936671072367
  }
]
```

5. Bonus

Phần Bonus yêu cầu tìm khoảng thời gian ngắn nhất (tính bằng giây) từ một thời điểm t đến khi tìm thấy một bản tin có giá cao hơn và một bản tin có giá thấp hơn trong vòng 20 giây tiếp theo. Để giải quyết bài toán này, nhóm đã sử dụng Spark Structured Streaming với phương pháp tự kết (self-join) trên luồng dữ liệu, một kỹ thuật nâng cao để phân tích tương quan thời gian.

5.1. Giải thích mã nguồn

5.1.1. Phương thức vận hành

1. Khởi tạo và cấu hình Spark Session

Đầu tiên, một `SparkSession` được khởi tạo để làm việc với YARN cluster. Các cấu hình quan trọng được thiết lập, bao gồm `appName` để định danh ứng dụng trên YARN UI, `spark.sql.shuffle.partitions` để tối ưu hóa hiệu năng.

Đặc biệt, `spark.sql.streaming.statefulOperator.checkCorrectness.enabled` được đặt thành `false`. Cấu hình này là cần thiết để cho phép các toán tử có trạng thái (stateful) phức tạp như `join` và `groupBy` có thể tạo ra các bản tin có timestamp trễ hơn so với watermark của luồng, điều không thể tránh khỏi trong logic của bài toán này.

2. Đọc và Tiền xử lý Luồng Dữ liệu từ Kafka

Ứng dụng “tham gia” (subscribe) vào Topic Kafka `btc-price` để nhận luồng dữ liệu giá BTC theo thời gian thực.

Mỗi bản tin nhận về từ Kafka (dưới dạng value) ban đầu là một chuỗi JSON.

Ta định nghĩa một schema (cấu trúc) rõ ràng bao gồm `symbol`, `price` và `timestamp` để parse chuỗi JSON này thành các cột dữ liệu có cấu trúc.

Cột `timestamp` sau đó được chuyển đổi từ kiểu chuỗi (string) sang kiểu timestamp của Spark để có thể thực hiện các phép toán về thời gian như watermarking và so sánh khoảng thời gian.

3. Thực hiện Self-Join với Watermarking.

Đây là bước quan trọng nhất để giải quyết bài toán. Ta tạo ra hai luồng dữ liệu tham chiếu (alias) là `left` và `right` từ cùng một luồng dữ liệu gốc. Sau đó, chúng em thực hiện một phép hợp trái (Left Outer Join) giữa hai luồng này.

• Điều kiện kết: Phép kết được thực hiện với ba điều kiện:

- `left.symbol = right.symbol`: Đây là điều kiện bằng bắt buộc của Spark cho stream-stream join, đảm bảo chỉ so sánh các bản tin có cùng mã giao dịch.
- `right.timestamp > left.timestamp`: Đảm bảo chúng ta chỉ tìm kiếm các bản tin xảy ra sau bản tin hiện tại.
- `right.timestamp <= left.timestamp + interval 20 seconds`: Giới hạn không gian tìm kiếm trong vòng 20 giây theo yêu cầu của đề bài.
- Watermarking: Để ngăn chặn việc trạng thái của phép hợp tăng lên vô hạn, ta áp dụng `withWatermark`.
 - Luồng `left` được gán watermark là 30 giây (20s của khoảng tìm kiếm + 10s lề cho dữ liệu trễ) và luồng `right` là 10 giây.
 - Watermarking cho phép Spark tự động dọn dẹp các trạng thái đã cũ, giúp ứng dụng chạy ổn định trong thời gian dài.

4. Tính toán Khoảng Thời gian Chính xác

Sau khi hợp, mỗi dòng ở `left` có thể được kết nối với nhiều dòng ở `right` (nếu có).

Để tính toán khoảng thời gian với độ chính xác đến mili-giây, ta chuyển đổi cả hai cột `left.timestamp` và `right.timestamp` sang kiểu `double`. Phép toán này giữ lại phần thập phân của giây, không giống như hàm `unix_timestamp()` sẽ làm tròn về giây và gây sai số.

Sau khi có được `time_diff_secs`, chúng em sử dụng `groupBy` theo `left.timestamp` và `left.symbol` để tìm ra `min()` (thời gian nhỏ nhất) cho các trường hợp giá cao hơn (`right.price > left.price`) và giá thấp hơn.

5. Xử lý Kết quả và Ghi vào Kafka

Kết quả của phép gom (aggregate) sẽ là `NULL` nếu không tìm thấy giá cao hơn (hoặc thấp hơn) trong cửa sổ 20 giây. Ta sử dụng hàm `coalesce` để thay thế các giá trị `NULL` này bằng giá trị mặc định là 20.0 theo yêu cầu đề bài.

Cuối cùng, ứng dụng tạo ra hai luồng kết quả riêng biệt: một cho `higher_window` và một cho `lower_window`. Mỗi luồng được định dạng lại thành chuỗi JSON và đẩy vào hai topic Kafka tương ứng là `btc-price-higher` và `btc-price-lower` để các hệ thống khác có thể tiêu thụ.

Việc ghi dữ liệu được thực hiện với `checkpointLocation` trỏ tới HDFS, đảm bảo tính chịu lỗi cho ứng dụng khi chạy trên cluster.

5.2. Thiết lập cá nhân ban đầu

- Kernel Ubuntu 24.04.2 LTS trên WSL2

```
(base) ngthinh@DESKTOP-UJ986V4: /home/ngthinh
→ wslfetch

./+00ssss00+/-./
`:+ssssssssssssssssss+:`
++ssssssssssssssssssyyssst+
.0sssssssssssssssssdMMNNy:ssso.
/sss:sssssssshdmmNNmyNNMMh:ssss/
+ssssssssshmydMMMMMMNdddy:ssssssst+
/sss:ssssshNNMMyhhyyyhmNNMMh:ssssss/
.sssssssdMMNNh:ssssssssshNNMMd:ssssssss.
+ssssshhyNNMMNy:sssssssssyNNMMy:ssssssst+
ossyNNMMNyMMh:ssssssssshmmh:ssssssso
ossyNNMMNyMMh:ssssssssshmmh:ssssssso
+ssssshhyNNMMNy:sssssssssyNNMMy:ssssssst+
.sssssssdMMNNh:ssssssssshNNMMd:ssssssss.
/sss:ssssshNNMMyhhyyyhdNNMMh:ssssss/
+ssssssssdmydMMMMMMNdddy:ssssssst+
/sss:sssssshdmmNNmyNNMMh:ssss/
.0sssssssssssssssssdMMNNy:ssso.
++ssssssssssssssssssyyssst+
`:+ssssssssssssssssss+:`
./+00ssss00+/-./

Windows Subsystem for Linux (WSL2)
ngthinh@DESKTOP-UJ986V4
Build: 19044
Branch: vb_release
Release: Ubuntu 24.04.2 LTS
Kernel: Linux 6.6.87.1-microsoft-standard-WSL2
Uptime: 0d 19h 17m
```

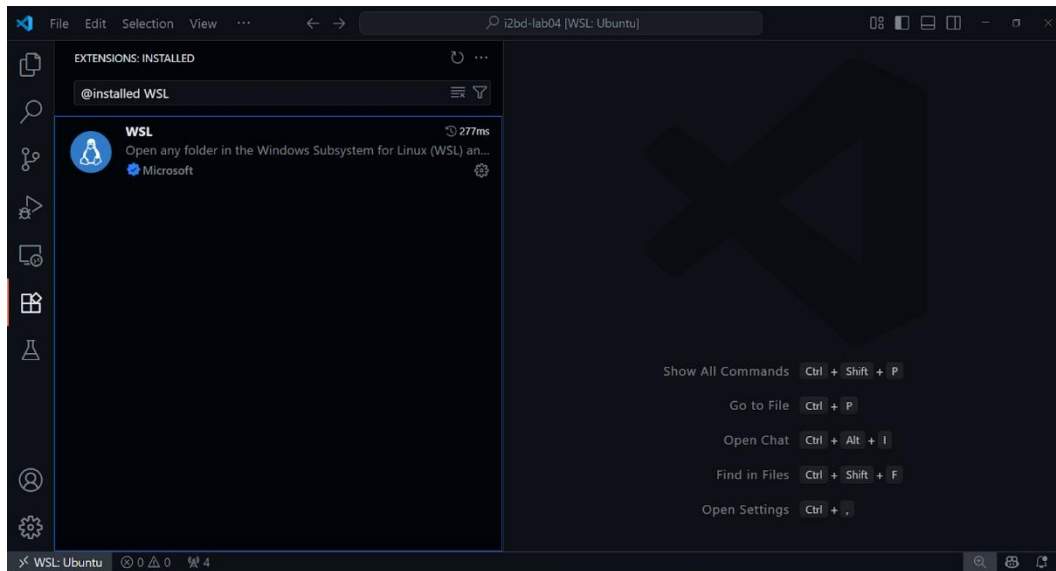
Hình 1: Thông tin chung của Kernel sử dụng câu lệnh `wslfetch`

- Conda 25.3.1 (Phiên bản Miniconda)

```
(base) ngthinh@DESKTOP-UJ986V4: /home/ngthinh
→ conda --version
conda 25.3.1
```

Hình 2: Phiên bản của conda sử dụng câu lệnh `conda --version`

- Visual Studio Code với extension WSL



Hình 3: Cấu hình Visual Studio Code với WSL

5.3. Cài đặt các thành phần cần thiết

5.3.1. Cài đặt và cấu hình Java và Hadoop

Việc cài đặt và cấu hình Java 8 (môi trường chạy những công cụ Big Data của Apache như Hadoop, Spark và Kafka) và Hadoop 3.4.1 (thiết lập và quản lý Hadoop Cluster) đã được trình bày trong Lab 01.

```
(base) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ java -version
openjdk version "1.8.0_452"
OpenJDK Runtime Environment (build 1.8.0_452-8u452-ga~us1-0ubuntu1~24.04-b09)
OpenJDK 64-Bit Server VM (build 25.452-b09, mixed mode)
```

Hình 4: Kiểm tra phiên bản Java

```
(base) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ tree ~/hadoop/hadoop-3.4.1 -d -L 1
/home/nqthinh/hadoop/hadoop-3.4.1
├── bin
├── etc
├── include
├── lib
├── libexec
├── licenses-binary
├── logs
├── sbin
└── share

10 directories
(base) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ hadoop version
Hadoop 3.4.1
Source code repository https://github.com/apache/hadoop.git -r 4d7825309348956336b8f06a08322b78422849b1
Compiled by mthakur on 2024-10-09T14:57Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum 7292fe9dba5e2e44e3a9f763fce3e680
This command was run using /home/nqthinh/hadoop/hadoop-3.4.1/share/hadoop/common/hadoop-common-3.4.1.jar
```

Hình 5: Đường dẫn và phiên bản của Hadoop trên Kernel

5.3.2. Cài đặt và cấu hình Kafka

Tổng quan về Apache Kafka:

- Apache Kafka là một nền tảng xử lý luồng dữ liệu phân tán, cho phép thu thập, xử lý và truyền tải dữ liệu theo thời gian thực với khả năng mở rộng cao. Kafka được sử dụng rộng rãi trong các hệ thống lớn để xây dựng các ứng dụng xử lý sự kiện và tích hợp dữ liệu hiệu quả.

Cài đặt và cấu hình Kafka:

- Tải file nén nhị phân của Kafka 3.9.1 dành cho Scala 2.13 từ trang web chính thức của Apache Kafka bằng wget, sau đó giải nén file vào thư mục ~/kafka bằng tar.

```
mkdir ~/kafka
cd ~/kafka
wget https://archive.apache.org/dist/kafka/3.9.1/kafka_2.13-3.9.1.tgz
tar -xvzf kafka_2.13-3.9.1.tgz
```

```
(base) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ tree ~/kafka/kafka_2.13-3.9.1 -d -L 1
/home/nqthinh/kafka/kafka_2.13-3.9.1
├── bin
├── config
├── libs
├── licenses
├── logs
└── site-docs

7 directories
(base) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ ~/kafka/kafka_2.13-3.9.1/bin/kafka-topics.sh --version
3.9.1
```

Hình 6: Đường dẫn và phiên bản của Kafka

5.3.3. Cài đặt và cấu hình PySpark

Tổng quan về Spark và PySpark:

- Apache Spark là một nền tảng xử lý dữ liệu lớn phân tán, hỗ trợ tính toán nhanh nhờ cơ chế in-memory và khả năng mở rộng linh hoạt. PySpark là API Python của Apache Spark, giúp lập trình viên Python dễ dàng khai thác sức mạnh của Spark trong xử lý dữ liệu và machine learning.
- PySpark là API Python cho Apache Spark, cho phép xử lý và phân tích dữ liệu lớn phân tán một cách hiệu quả và nhanh chóng. PySpark hỗ trợ các thao tác xử lý dữ liệu theo mô hình RDD và DataFrame, rất phổ biến trong các bài toán big data và machine learning.

Cài đặt và cấu hình PySpark:

- Để đảm bảo môi trường Python dùng cho Lab này không bị ảnh hưởng bởi các cấu hình ngoài, ta tạo một môi trường conda mới tên là để chứa các thư viện của Lab.

```
conda create -n i2bd-env
conda activate i2bd-env
```

- Sau đó, ta cài đặt thư viện PySpark phiên bản 3.2.4 (tương thích với Java 8) thông qua trình quản lý gói của Python 3 (pip).

```
pip install pyspark==3.2.4
```

```
(i2bd-env) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ conda info | grep -E 'active environment|python version'
    active environment : i2bd-env
    python version      : 3.13.2.final.0
(i2bd-env) nqthinh@DESKTOP-UJ986V4: /home/nqthinh
→ pip show pyspark
Name: pyspark
Version: 3.2.4
Summary: Apache Spark Python API
Home-page: https://github.com/apache/spark/tree/master/python
Author: Spark Developers
Author-email: dev@spark.apache.org
License: http://www.apache.org/licenses/LICENSE-2.0
Location: /home/nqthinh/softwares/softwares/miniconda3/envs/i2bd-env/lib/python3.11/site-packages
Requires: py4j
Required-by:
```

Hình 7: Thông tin tổng quát của thư viện PySpark theo pip

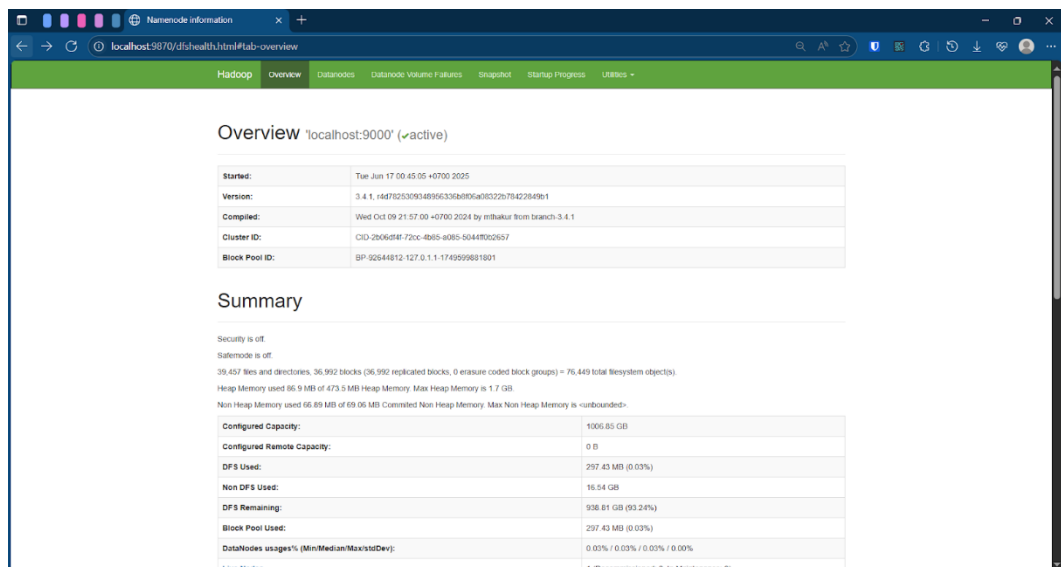
5.4. Khởi động hệ thống

5.4.1. Khởi động HDFS và YARN trong Hadoop.

Khởi động HDFS và YARN bằng những câu lệnh sau:

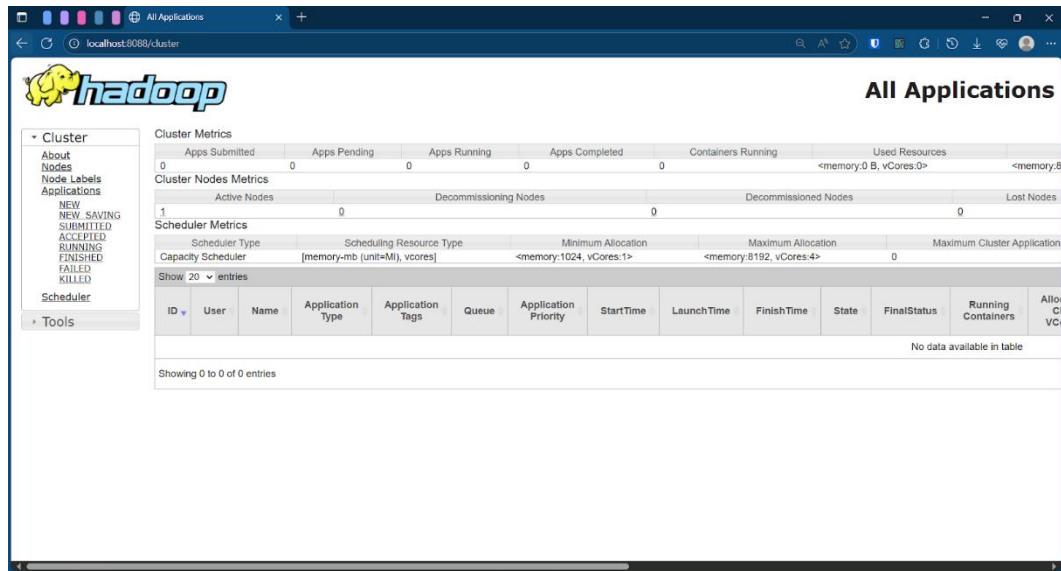
```
cd ~/hadoop/hadoop-3.4.1
./sbin/start-yarn.sh && ./sbin/start-dfs.sh
```

Web UI của HDFS ở cổng 9870:



Hình 8: Hadoop UI

Web UI của YARN ở cổng 8088



Hình 9: YARN UI

5.4.2. Chạy Kafka Zookeeper và Kafka Broker

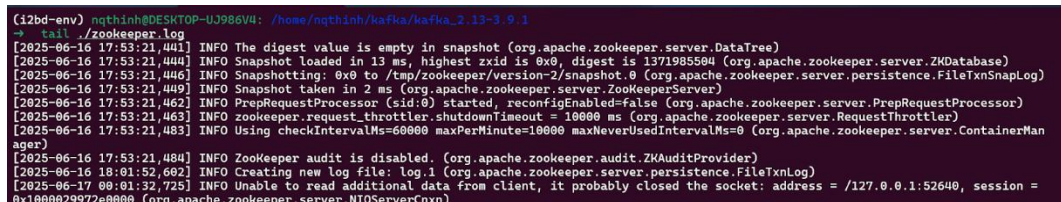
Chạy Kafka Zookeeper, một dịch vụ điều phối giúp Kafka quản lý cụm broker, theo dõi trạng thái và duy trì tính nhất quán trong hệ thống phân tán. File .configurations của Kafka Zookeeper được sử dụng ở đây là file mặc định, chưa qua sửa đổi.

```
cd ~/kafka/kafka_2.13-3.9.1
./bin/zookeeper-server-start.sh. /config/zookeeper.properties
./zookeeper.log 2> ./err_zookeeper.log
```

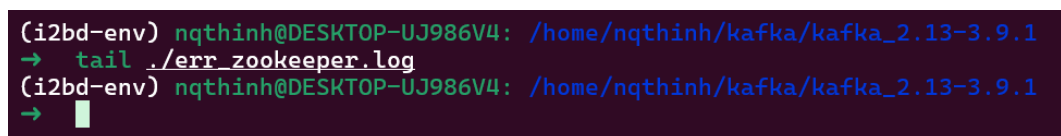


Hình 10: Chạy Server Zookeeper

Ở đây, ta chuyển luồng xuất ra màn hình của Zookeeper vào file zookeeper.log, và chuyển lỗi xuất lỗi ra màn hình vào file err_zookeeper.log



Hình 11: In Logging stdout của Kafka Zookeeper



Hình 12: In logging lỗi của Kafka Zookeeper

Việc file err_zookeeper.log rỗng cho thấy phần nào Kafka Zookeeper đang hoạt động tốt.

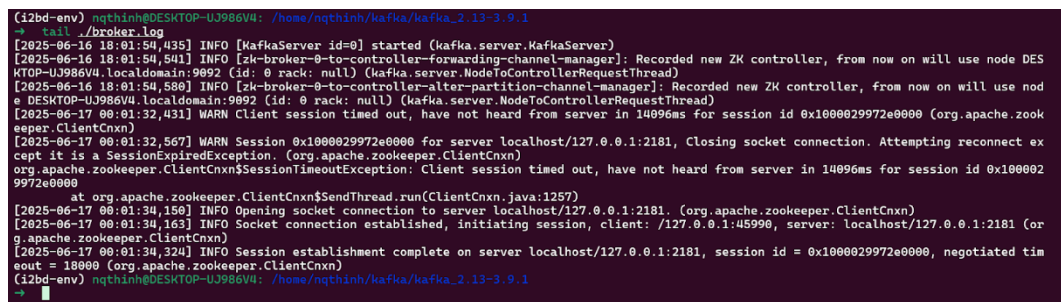
Sau đó, ta chạy Kafka Broker, là thành phần chịu trách nhiệm nhận, lưu trữ và phân phối dữ liệu giữa producer và consumer trong hệ thống Kafka. File .configurations của Kafka Broker được sử dụng ở đây là file mặc định, chưa qua sửa đổi.

```
cd ~/kafka/kafka_2.13-3.9.1
./bin/kafka-server-start.sh ./config/server.properties > ./broker.log
2> ./err_broker.log
```

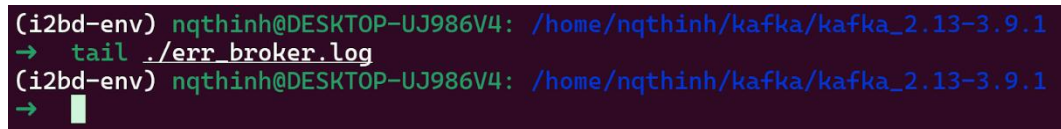


Hình 13: Chạy Kafka Server

Ở đây, ta chuyển luồng xuất ra màn hình của Broker vào file broker.log, và chuyển lỗi xuất lỗi ra màn hình vào file err_broker.log



Hình 14: In Logging stdout của Kafka Server



Hình 15: In Logging lỗi của Kafka Server

Việc file err_broker.log rỗng cho thấy phần nào Kafka Broker đang hoạt động tốt.

5.4.3. Tạo Kafka Topic cho high_price và low_price

Kafka Topic là kênh logic dùng để phân loại và lưu trữ luồng dữ liệu, nơi mà producer gửi và consumer nhận message.

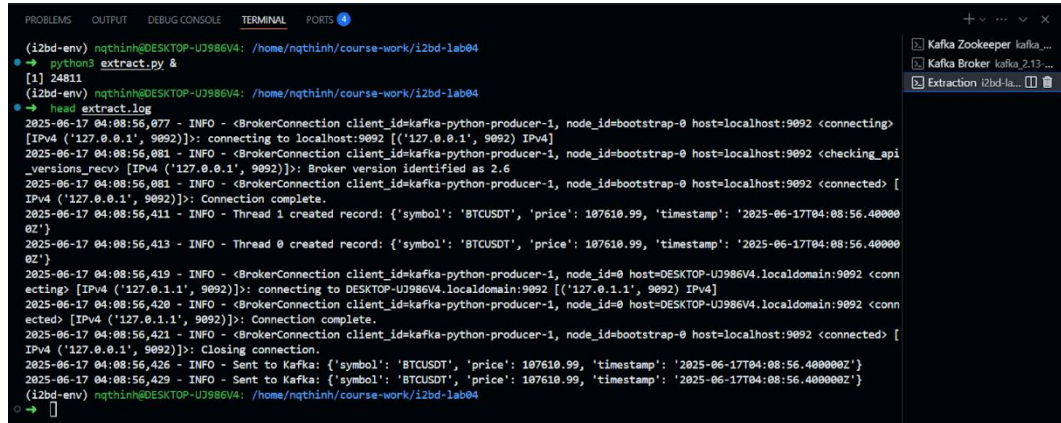
Ta tạo hai Kafka Topic cho việc tìm mức giá cao hơn và thấp hơn so với mức giá hiện tại đang xét lần lượt là btc-price-higher và btc-price-lower (kết hợp với Kafka Topic btc-price ở giai đoạn Extract) để gửi và nhận thông tin liên tục tại cổng 9092.

```
~/kafka/kafka_2.13-3.6.1/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --topic btc-price-higher --partitions 1 --
replication-factor 1
~/kafka/kafka_2.13-3.6.1/bin/kafka-topics.sh --bootstrap-server
localhost:9092 --create --topic btc-price-lower --partitions 1 --
replication-factor 1
```

5.4.4. Chạy Pipeline dữ liệu

Chạy file `extract.py` từ giai đoạn Extract. Kết quả của giai đoạn này được lưu trữ trong file `extract.log` ở cùng cấp thư mục với file mã nguồn.

```
python3 extract.py
```



```
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
→ python3 extract.py &
[1] 24811
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
→ head extract.log
2025-06-17 04:08:56,077 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <connecting> [IPv4 ('127.0.0.1', 9092)]>: connecting to localhost:9092 [IPv4 ('127.0.0.1', 9092) IPv4]
2025-06-17 04:08:56,081 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <checking_api_versions_recv> [IPv4 ('127.0.0.1', 9092)]>: Broker version identified as 2.6
2025-06-17 04:08:56,081 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <connected> [IPv4 ('127.0.0.1', 9092)]>: Connection complete.
2025-06-17 04:08:56,411 - INFO - Thread 1 created record: {'symbol': 'BTCUSD', 'price': 107610.99, 'timestamp': '2025-06-17T04:08:56.400000Z'}
2025-06-17 04:08:56,413 - INFO - Thread 0 created record: {'symbol': 'BTCUSD', 'price': 107610.99, 'timestamp': '2025-06-17T04:08:56.400000Z'}
2025-06-17 04:08:56,419 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=0 host=DESKTOP-UJ986V4.localdomain:9092 <connecting> [IPv4 ('127.0.0.1', 9092)]>: connecting to DESKTOP-UJ986V4.localdomain:9092 [IPv4 ('127.0.0.1', 9092) IPv4]
2025-06-17 04:08:56,420 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=0 host=DESKTOP-UJ986V4.localdomain:9092 <connected> [IPv4 ('127.0.0.1', 9092)]>: Connection complete.
2025-06-17 04:08:56,421 - INFO - <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=localhost:9092 <connected> [IPv4 ('127.0.0.1', 9092)]>: Closing connection.
2025-06-17 04:08:56,426 - INFO - Sent to Kafka: {'symbol': 'BTCUSD', 'price': 107610.99, 'timestamp': '2025-06-17T04:08:56.400000Z'}
2025-06-17 04:08:56,429 - INFO - Sent to Kafka: {'symbol': 'BTCUSD', 'price': 107610.99, 'timestamp': '2025-06-17T04:08:56.400000Z'}
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
→
```

Hình 16: Chạy `extract.py` và kiểm tra luồng của Kafka Topic `btc-price`

Định dạng giá trị vào được là một object dưới dạng JSON với định dạng.

```
{
  "symbol": <a string>,
  "price": <a floating-point value>
  "timestamp": <a string of the current time following the ISO8601 standards>
}
```

Gửi công việc Spark lên YARN bằng `spark-submit` (được cài đặt cùng `pyspark`) để xử lý `btc-price` và xuất đầu ra phù hợp cho từng luồng của các Topic.

```
spark-submit \
  --master yarn \
  --deploy-mode client \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.2.4 \
  bonus.py > bonus.log 2> err_bonus.log &
```

Ở đây, ta chuyển luồng xuất ra màn hình của Spark vào file `bonus.log`, và chuyển lỗi xuất lỗi ra màn hình vào file `err_bonus.log`

```

(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
➔ spark-submit \
--master yarn \
--deploy-mode client \
--packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.2.4 \
bonus.py > bonus.log &
[1] 26809
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
➔ tail -f bonus.log
[1] + 26809 exit 1 spark-submit --master yarn --deploy-mode client --packages bonus.py > &
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
➔ tail -f bonus.log
:
+ StreamingDataSourceV2Relation [key#7, value#8, topic#9, partition#10, offset#11, timestamp#12, ti
mestampType#13], org.apache.spark.sql.kafka010.KafkaSourceProvider$KafkaScan@6c8a1b84, KafkaV2[Subscribe[btc-price]]
:
- CTERelationDef 1
:
+ SubqueryAlias first_price_movements
:
+ Aggregate [event_timestamp#32-T30000ms, symbol#23, event_price#33], [event_timestamp#32-T30000ms, symbol#23, event_price#3
3, min(CASE WHEN (future_price#35 > event_price#33) THEN future_timestamp#34-T30000ms END) AS first_higher_timestamp#36, min(CASE WHEN (f
uture_price#35 < event_price#33) THEN future_timestamp#34-T30000ms END) AS first_lower_timestamp#37]
:
+ SubqueryAlias future_price_events
:
+ CTERelationRef 0, true, [event_timestamp#32-T30000ms, symbol#23, event_price#33, future_timestamp#34-T30000ms, futur
e_price#35]
:
+ Project [event_timestamp#32-T30000ms AS timestamp#29-T30000ms, coalesce(cast((unix_timestamp(first_higher_timestamp#36, yyyy-MM-
dd HH:mm:ss, Some(Etc/UTC), false) - unix_timestamp(event_timestamp#32-T30000ms, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false)) as double),
cast(20.0 as double)) AS higher_window#30, coalesce(cast((unix_timestamp(first_lower_timestamp#37, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), fa
lse) - unix_timestamp(event_timestamp#32-T30000ms, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), false)) as double), cast(20.0 as double)) AS lower
_window#31]
:
+ SubqueryAlias first_price_movements
:
+ CTERelationRef 1, true, [event_timestamp#32-T30000ms, symbol#23, event_price#33, first_higher_timestamp#36, first_lower_ti
mestamp#37]

```

Hình 17: Chạy bonus.py trong nền và in ra file logging stdout của app Spark Streaming

Ở file err_bonus.log, đa số các lỗi đều ở mức INFO và WARN, bao gồm:

- Việc không có thư viện native cho các dịch vụ của Apache, nên sẽ sử dụng Java 8 để chạy thay thế.
- Việc một số thư mục chứa mã nguồn của công cụ bị trùng hoặc thiếu, cần cài đặt thêm.

Cuối cùng, ta xem kết quả qua các luồng của các Topic.

```

# Ghi log cửa sổ giá cao hơn vào file high_prices.log
~/kafka/kafka_2.13-3.9.1/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic btc-price-higher > high_prices.log &

# Ghi log cửa sổ giá thấp hơn vào file low_prices.log
~/kafka/kafka_2.13-3.9.1/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic btc-price-lower > low_prices.log &

```

```

(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
➔ ~/kafka/kafka_2.13-3.9.1/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic btc-price-higher > high_prices.log &
[1] 48547
[2] 48548
(i2bd-env) ngthinh@DESKTOP-UJ986V4: /home/ngthinh/course-work/i2bd-lab04
➔

```

Hình 18: Chuyển hướng hai luồng của hai Kafka Topic btc-price-higher và btc-price-lower vào hai file logging lần lượt là high_prices.log và low_prices.log

Application application_1750150507979_0002

Application Overview

User: oatfish
 Name: bonus.py
 Application Type: SPARK
 Application Priority: 0 (higher integer value indicates higher priority)
 YarnApplicationState: RUNNING: AM has registered with RM and started running.
 Queue: root-default
 FinalStatus Reported by AM: Application has not completed yet.
 Started: Tue Jun 17 09:08:03 +0000 2025
 Launched: Tue Jun 17 09:08:03 +0000 2025
 Finished: N/A
 Elapsed: 25mins, 50secs
 Tracking URL: ApplicationMaster
 Log Aggregation Status: DISABLED
 Application Timeout (Remaining Time): Unlimited
 Diagnostics: Unlimited
 Unmanaged Application: false
 Application Node Label expression: <Not set>
 AM container Node Label expression: <DEFAULT_PARTITION>

Application Metrics

Total Resource Preempted: <memory 0, vCores 0>
 Total Number of Non-AM Containers Preempted: 0
 Total Number of AM Containers Preempted: 0
 Resource Preempted from Current Attempt: <memory 0, vCores 0>
 Number of Non-AM Containers Preempted from Current Attempt: 0
 Aggregate Resource Allocation: 12590354 MB-seconds, 4617 vcore-seconds
 Aggregate Preempted Resource Allocation: 0 MB-seconds, 0 vcore-seconds

Attempt ID	Started	Node	Logs	Nodes blacklisted by the app	Nodes blacklisted by the system
application_1750150507979_0002_000001	Tue Jun 17 16:08:03 +0700 2025	hdp://DESKTOP-UJ956V4.localdomain:8042	Logs	0	0

Hình 19: Thông tin của công việc của app Spark Streaming của bonus trên YARN UI

high_prices.log

```

579 ["timestamp": "2025-06-17T09:25:31.000Z", "higher_window": "11.0"]
580 ["timestamp": "2025-06-17T09:25:30.800Z", "higher_window": "10.0"]
581 ["timestamp": "2025-06-17T09:25:30.600Z", "higher_window": "12.0"]
582 ["timestamp": "2025-06-17T09:25:30.400Z", "higher_window": "14.0"]
583 ["timestamp": "2025-06-17T09:25:30.200Z", "higher_window": "12.0"]
584 ["timestamp": "2025-06-17T09:25:28.300Z", "higher_window": "14.0"]
585 ["timestamp": "2025-06-17T09:25:31.700Z", "higher_window": "11.0"]
586 ["timestamp": "2025-06-17T09:25:28.200Z", "higher_window": "14.0"]
587 ["timestamp": "2025-06-17T09:25:30.100Z", "higher_window": "12.0"]
588 ["timestamp": "2025-06-17T09:25:29.900Z", "higher_window": "11.0"]
589 ["timestamp": "2025-06-17T09:25:28.700Z", "higher_window": "14.0"]
590 ["timestamp": "2025-06-17T09:25:30.600Z", "higher_window": "12.0"]
591 ["timestamp": "2025-06-17T09:25:29.600Z", "higher_window": "13.0"]
592 ["timestamp": "2025-06-17T09:25:34.500Z", "higher_window": "11.0"]
593 ["timestamp": "2025-06-17T09:25:34.300Z", "higher_window": "11.0"]
594 ["timestamp": "2025-06-17T09:25:34.000Z", "higher_window": "11.0"]
595 ["timestamp": "2025-06-17T09:25:33.200Z", "higher_window": "10.0"]
596 ["timestamp": "2025-06-17T09:25:35.900Z", "higher_window": "20.0"]
597 ["timestamp": "2025-06-17T09:25:35.000Z", "higher_window": "10.0"]
598 ["timestamp": "2025-06-17T09:25:35.600Z", "higher_window": "10.0"]
599 ["timestamp": "2025-06-17T09:25:32.400Z", "higher_window": "10.0"]
600 ["timestamp": "2025-06-17T09:25:33.700Z", "higher_window": "12.0"]
601 ["timestamp": "2025-06-17T09:25:35.800Z", "higher_window": "20.0"]
602 ["timestamp": "2025-06-17T09:25:36.100Z", "higher_window": "20.0"]
603 ["timestamp": "2025-06-17T09:25:35.300Z", "higher_window": "10.0"]
604 ["timestamp": "2025-06-17T09:25:34.800Z", "higher_window": "11.0"]
605 ["timestamp": "2025-06-17T09:25:34.900Z", "higher_window": "11.0"]
606 ["timestamp": "2025-06-17T09:25:35.400Z", "higher_window": "10.0"]
607 ["timestamp": "2025-06-17T09:25:33.100Z", "higher_window": "10.0"]
608 ["timestamp": "2025-06-17T09:25:35.100Z", "higher_window": "20.0"]
609 ["timestamp": "2025-06-17T09:25:34.100Z", "higher_window": "11.0"]
610 ["timestamp": "2025-06-17T09:25:32.000Z", "higher_window": "10.0"]
611 ["timestamp": "2025-06-17T09:25:36.000Z", "higher_window": "20.0"]
612 ["timestamp": "2025-06-17T09:25:35.500Z", "higher_window": "10.0"]
613 ["timestamp": "2025-06-17T09:25:35.700Z", "higher_window": "20.0"]
614 ["timestamp": "2025-06-17T09:25:33.900Z", "higher_window": "12.0"]
615 ["timestamp": "2025-06-17T09:25:32.500Z", "higher_window": "10.0"]

```

low_prices.log

```

558 ["timestamp": "2025-06-17T09:25:27.700Z", "lower_window": "11.0"]
559 ["timestamp": "2025-06-17T09:25:30.800Z", "lower_window": "10.0"]
560 ["timestamp": "2025-06-17T09:25:30.800Z", "lower_window": "10.0"]
561 ["timestamp": "2025-06-17T09:25:28.600Z", "lower_window": "12.0"]
562 ["timestamp": "2025-06-17T09:25:30.700Z", "lower_window": "10.0"]
563 ["timestamp": "2025-06-17T09:25:28.300Z", "lower_window": "14.0"]
564 ["timestamp": "2025-06-17T09:25:28.200Z", "lower_window": "12.0"]
565 ["timestamp": "2025-06-17T09:25:30.100Z", "lower_window": "10.0"]
566 ["timestamp": "2025-06-17T09:25:27.100Z", "lower_window": "14.0"]
567 ["timestamp": "2025-06-17T09:25:29.900Z", "lower_window": "14.0"]
568 ["timestamp": "2025-06-17T09:25:27.200Z", "lower_window": "14.0"]
569 ["timestamp": "2025-06-17T09:25:28.700Z", "lower_window": "12.0"]
570 ["timestamp": "2025-06-17T09:25:30.600Z", "lower_window": "10.0"]
571 ["timestamp": "2025-06-17T09:25:29.600Z", "lower_window": "11.0"]
572 ["timestamp": "2025-06-17T09:25:33.200Z", "lower_window": "15.0"]
573 ["timestamp": "2025-06-17T09:25:33.600Z", "lower_window": "17.0"]
574 ["timestamp": "2025-06-17T09:25:32.400Z", "lower_window": "16.0"]
575 ["timestamp": "2025-06-17T09:25:33.700Z", "lower_window": "15.0"]
576 ["timestamp": "2025-06-17T09:25:33.500Z", "lower_window": "10.0"]
577 ["timestamp": "2025-06-17T09:25:33.000Z", "lower_window": "15.0"]
578 ["timestamp": "2025-06-17T09:25:33.700Z", "lower_window": "17.0"]
579 ["timestamp": "2025-06-17T09:25:33.900Z", "lower_window": "15.0"]
580 ["timestamp": "2025-06-17T09:25:32.500Z", "lower_window": "16.0"]
581 ["timestamp": "2025-06-17T09:25:31.900Z", "lower_window": "17.0"]
582 ["timestamp": "2025-06-17T09:25:32.800Z", "lower_window": "16.0"]
583 ["timestamp": "2025-06-17T09:25:32.800Z", "lower_window": "16.0"]
584 ["timestamp": "2025-06-17T09:25:34.000Z", "lower_window": "14.0"]
585 ["timestamp": "2025-06-17T09:25:33.400Z", "lower_window": "10.0"]
586 ["timestamp": "2025-06-17T09:25:33.800Z", "lower_window": "15.0"]
587 ["timestamp": "2025-06-17T09:25:33.100Z", "lower_window": "15.0"]
588 ["timestamp": "2025-06-17T09:25:32.900Z", "lower_window": "16.0"]
589 ["timestamp": "2025-06-17T09:25:31.800Z", "lower_window": "17.0"]
590 ["timestamp": "2025-06-17T09:25:32.300Z", "lower_window": "16.0"]
591 ["timestamp": "2025-06-17T09:25:32.100Z", "lower_window": "16.0"]
592 ["timestamp": "2025-06-17T09:25:32.200Z", "lower_window": "16.0"]
593 ["timestamp": "2025-06-17T09:25:32.600Z", "lower_window": "16.0"]
594 ["timestamp": "2025-06-17T09:25:32.700Z", "lower_window": "16.0"]

```

Hình 20: Một số entry của hai file logging high_prices.log và low_prices.log

Khi ta muốn kết thúc, ta tắt tất cả các terminal còn hoạt động, bao gồm:

- Spark-submit chạy bonus.py
- Python 3 chạy extract.py
- Kafka Server (cho Broker)
- Kafka Zookeeper

Đồng thời, ta cũng tắt cả HDFS và YARN bằng câu lệnh lần lượt là ./stop-dfs.sh và ./stop-yarn.sh.

Tài liệu tham khảo

- [1] [Online]. Available: <https://www.confluent.io/blog/set-up-and-run-kafka-on-windows-linux-wsl-2>.
- [2] [Online]. Available: <https://spark.apache.org/docs/latest/streaming/structured-streaming-kafka-integration.html>.
- [3] [Online]. Available: <https://sparkbyexamples.com/pyspark/pyspark-sql-with-examples/>.
- [4] [Online]. Available: <https://www.sparkcodehub.com/pyspark/streaming/watermarking>.
- [5] [Online]. Available: <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-community-with-docker/>.